

Anchor reorg-of-reorg recovery: three changes shipped, one for your call

Andreas Kohl · Sequentia testnet · 2026-06-28 · commit `23b1be0b1` (branch `claude/sequentia-bitcoin-sidechain-w6xady`)

This follows up our exchange about the recovery gap you endorsed fixing: on a Bitcoin reorg-of-reorg ($A \rightarrow B \rightarrow A$, routine on testnet4), Sequentia blocks anchored to A that the node invalidated when Bitcoin left A must be **re-validated and re-finalized** when Bitcoin returns to A, not replaced by a fresh block on top. As you noted, minting fresh would let an atomic swap that is valid again on the Bitcoin side be double-spent on the Sequentia side.

Three changes are implemented, reviewed (two adversarial passes), tested, and deployed to the testnet committee. A fourth, distinct issue surfaced during the work and needs a consensus-rule decision from you; it is described last. None of the three shipped changes alters a consensus rule: the new state is node-local, so old and new binaries are consensus-compatible.

The defect

The anchor watcher (`AnchorWatchTask`, `src/anchor.cpp`) has two halves. *Invalidation* walks the active chain every tick and, from live Bitcoin ground truth, invalidates any block whose anchor is off Bitcoin's best chain. *Recovery* was asymmetric: it only iterated an in-memory set of blocks the watcher had invalidated, and only when the parent tip changed. The `BLOCK_FAILED` flags persist across a restart; that in-memory set does not. So a restart (or a missed/coalesced parent flap) between the invalidation and the anchor's return left the previously valid-and-finalized blocks `BLOCK_FAILED` forever, and a branch known only as headers never re-requested its bodies.

The three changes SHIPPED

1. A recovery worklist that survives a restart

At startup, `BlockManager::LoadBlockIndex` re-seeds the recovery worklist from the persisted block index (new `SeedAnchorInvalidated`), and the reconsider loop now runs whenever the worklist is non-empty rather than only on a parent tip change. Together these recover the blocks after a restart and after a coalesced flap. Reconsideration still clears a block only when `checkMainchainAnchor` returns OK on a live parent-chain check (≥ 1 confirmation on the parent best chain at the claimed height); it never acts on `NOT_FOUND` / `STALE` / `NO_CONNECTION`.

2. A provenance marker so only anchor invalidations are recovered

This one is load-bearing for safety. `BLOCK_FAILED_VALID` records *that* a block failed, never *why*: the

anchor watcher, the `invalidateblock` RPC, and consensus connect-failures all set the same flag, and every anchored block carries an anchor. An earlier version inferred "anchor-invalidated" from `BLOCK_FAILED_VALID` + an anchor being present; an adversarial review showed that would **resurrect operator- and consensus-invalidated blocks on restart** (their anchor was never the problem, so it checks OK and gets reconsidered). That would silently undo the documented finality-split recovery (an operator `invalidateblock`-ing a stuck tip) and break the standard guarantee that `invalidateblock` persists across a restart.

The fix is a dedicated node-local status bit `BLOCK_FAILED_ANCHOR = 512` (`chain.h`), kept **outside** `BLOCK_FAILED_MASK` so it is pure provenance, set only by the watcher's invalidation (`CChainState::MarkAnchorInvalid`) and cleared in `ResetBlockFailureFlags`. Seeding and recovery consider only marked blocks; manual and consensus invalidations stay durably invalid.

3. Best-header recovery, without an RPC storm

After clearing the failure flags, `RecalculateBestHeader` raises `pindexBestHeader` onto the recovered branch (highest `BLOCK_VALID_TREE` block, which excludes any failed block), so a branch held only as headers re-requests its bodies; `ActivateBestChain` + `UpdateTip` then reconnect and re-finalize the whole restored chain, exactly as you required, rather than minting fresh. Because the reconsider loop now runs every tick, a negative anchor cache (`g_anchor_stale_cache`, cleared on a parent tip change like the existing OK cache) ensures each orphaned anchor is checked against bitcoind at most once per parent-tip epoch, and the loop bails on `NO_CONNECTION` rather than hammering a busy parent daemon.

Safety. Anchoring supremacy is preserved by construction. A block is un-failed only when its stored anchor returns OK on a live parent-chain check; the invalidation half keeps re-deriving bad-ness from ground truth every tick, so a block reconsidered just before its anchor re-orphan is re-invalidated on the next tick. `RecalculateBestHeader` uses `BLOCK_VALID_TREE`, so best-header can never land on a failed/orphaned branch, and `UpdateTip` remains the only finalizer, recomputing the finalized point from the resulting active chain. Files: `chain.h`, `anchor.{h,cpp}`, `validation.{h,cpp}`, `node/blockstorage.cpp`. Test: `feature_pos_reorg_of_reorg_recovery.py` (recovery across a restart restores the original blocks verbatim; a manual `invalidateblock` is not resurrected across a restart).

The fourth change: committee-equivocation finality split NEEDS YOUR CALL

This is distinct from the reorg-of-reorg case and is a genuine consensus-rule question, so I did not implement it. It is the 96/4 split we saw live: two *different* blocks at the same height, both anchored to the *same* valid Bitcoin block, each reaching quorum among a disjoint subset of the committee. The minority's finalized block has an intact anchor, so the immediate-finality gate's anchor-release valve stays false and it rejects the longer rival chain (*bad-fork-prior-to-pos-final*), and `AcceptBlockHeader` bails before storing it, so the rival is held only as headers. Anchoring cannot break this tie because both anchors are equally valid.

The shape I would propose, for your judgement: when a rival branch is (i) anchored to the current best Bitcoin chain and (ii) *strictly better committee-certified* than the local immediate-finalized block, the gate yields, the header is stored, and the existing comparator (more countersigs, then lower VRF) plus `ActivateBestChain` switch to it, with `UpdateTip` re-finalizing on the winner. The comparator already encodes the tie-break; the only blockers are the accept-time gate and the not-stored bail.

Questions for you:

1. Is un-finalizing a quorum-certified block ever admissible in favour of a same-anchor, strictly-better-certified sibling? If so, what is the exact deterministic predicate: countersig count only, or plus VRF, or plus chainwork?
2. If the two siblings carry *equal* certification (true equivocation from disjoint subgroups), the comparator falls through to VRF / first-seen. Do we need an explicit equivocation rule there, and what should it be?
3. Guardrails I would keep regardless: never yield across the Bitcoin checkpoint floor, and keep the rejection soft (no peer banning). Are those sufficient in your view?

Interim operational handling until this lands: an operator `invalidateblocks` the stuck minority tip (now durable across restart thanks to change 2). The live 96/4 split this week resolved itself when a deep testnet4 reorg rolled the chain back below the split point, but the gap remains. Full design notes and the open questions are in `doc/sequentia/anchor-reorg-of-reorg-recovery-design.md`.